

Tasca S5.01. Consultes amb MongoDB

IT ACADEMY



Danna Vallès

Peer to Peer (2P2):

Data Engineering MongoDB: Data Pipeline Analítica en Base de Datos NoSQL

Diseñar e implementar un **Data Lakehouse** básico sobre MongoDB para una aplicación de entretenimiento cinematográfico.

Se trabajará con un modelo de colecciones semi-estructuradas, aplicando técnicas de **ingestión, modelado de datos documentales, indexación y consultas analíticas** avanzadas para demostrar capacidad en la capa analítica (Data Engineering + Analytics).

Colecciones principales (Raw Layer):

- **users**: Perfiles de usuarios (nombre, email, password hashed).
- **theatres**: Información geoespacial de cines (dirección, coordenadas 2dsphere).
- **sessions**: Gestión de sesiones y autenticación (JWT tokens).
- **movies**: Catálogo rico de películas (título, plot, genres array, cast array, directors, year, runtime, awards, imdb rating, language, country, etc.).
- **comments**: Interacciones usuario-contenido (author, movie_id, text, date).

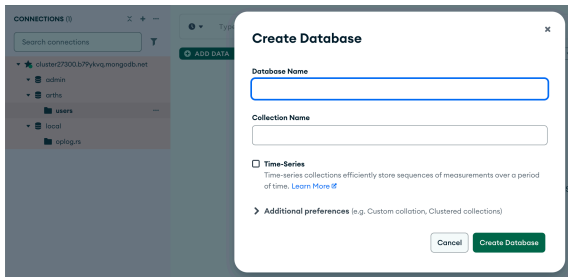
Aspectos Técnicos Clave -Data Engineering

- **Modelo de Datos**: Documental (denormalizado) vs relacional. Ventajas en lecturas y flexibilidad para catálogos ricos (arrays de géneros, cast, awards).
- **Medallion Architecture** (simplificada):
 - **Bronze (Raw Layer)**: Datos crudos importados → JSON, sin transformación.
 - estructura original con todos sus tipos, inconsistencias y campos opcionales. `arths.movies`, `arths.comments`, `arths.users`, `arths.theaters`, `arths.sessions`
 - **Silver (Curated Layer)**: Posibles vistas materializadas o colecciones enriquecidas (ej. películas con conteo de comentarios).
 - **Gold (Analytical Layer)**: Agregaciones y vistas para consumo analítico - Dashboard o Reportines

Configuración Inicial Del Entorno De Trabajo

"Create Database" dos campos:

- **Database Name:**
- **Collection Name:** `users`



O

"Open MongoDB shell":

```
use arths
```

```
db.createCollection("users") // Para crear la base arths con la colección users directamente.
```

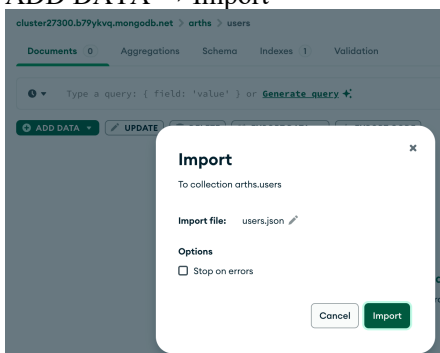
```
db.createCollection("comments")
```

```
db.createCollection("theaters")
```

```
db.createCollection("movies")
```

```
show collections // comprobar si las colecciones existen.
```

ADD DATA → Import



Users #185

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

cluster27300.b79ykvq.mongodb.net

admin

arths

comments

movies

sessions

theaters

users

local

cluster27300.b79ykvq.mongodb.net

users

comments

movies

sessions

theaters

users

mongodb: cluster27300...

cluster27300.b79ykvq.mongodb.net

arths

users

Documents185

Aggregations

Schema

Indexes1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ExplainResetFindOptions

ADD DATAUPDATEDELETEEXPORT DATAEXPORT CODE

251 - 25 of 185

_id: ObjectId('59b99db4cfa9a34cd7885b6')

name: "Ned Stark"

email: "sean_bean@gameofthron.es"

password: "\$2b\$12\$UREFwsRUoyF0CRqGNK0Lz00HM/jLhgUCNNI39R3AqMUQ74crL31Vu"

_id: ObjectId('59b99db4cfa9a34cd7885b7')

name: "Robert Baratheon"

email: "mark_add@gameofthron.es"

password: "\$2b\$12\$yGqXL69LZpXA2xVDhuPn50Zd.VURVxz7wg0LY3pn00s7u2S1Z032y"

_id: ObjectId('59b99db5cfa9a34cd7885b8')

name: "Jaime Lannister"

email: "nikolaj_coster-waldau@gameofthron.es"

password: "\$2b\$12\$6vz7w1w0.E1SR1lvq1zUc./9480gb1uPtXcahDxIadgyC3PS8XCUK"

_id: ObjectId('59b99db5cfa9a34cd7885b9')

name: "Catelyn Stark"

email: "michelle_fairley@gameofthron.es"

password: "\$2b\$12\$fiTH5Sh1zKNFX2i/FTEreW6jxo3xvmV7XL.q1fqCr8Cw0xK.nZW5"

_id: ObjectId('59b99db6cfa9a34cd7885ba')

name: "Cersei Lannister"

email: "Lena_headey@gameofthron.es"

Import completed.

1564 documents imported.

Comments #50k

cluster27300.b79ykvq.mongodb.net

arths

comments

Documents50K

Aggregations

Schema

Indexes1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ExplainResetFindOptions

ADD DATAUPDATEDELETEEXPORT DATAEXPORT CODE

25

_id: ObjectId('5a9427648b0beeb6b69579cc')

name: "Andrea Le"

email: "andrea_le@fakegmail.com"

movie_id: ObjectId('573a1390f29313caabdc418c')

text: "Rem officiiis eaque repellendus amet eos doloribus. Porro dolor volupta..."

date: 2012-03-26T23:20:16.000+00:00

_id: ObjectId('5a9427648b0beeb6b69579cf')

name: "Greg Powell"

email: "greg_powell@fakegmail.com"

movie_id: ObjectId('573a1390f29313caabdc41b1')

text: "Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Co..."

date: 1987-02-10T00:29:36.000+00:00

_id: ObjectId('5a9427648b0beeb6b69579d0')

name: "Talisa Maegyr"

email: "oona_chaplin@gameofthron.es"

movie_id: ObjectId('573a1390f29313caabdc41b1')

text: "Rem itaque ad sit rem voluptatibus. Ad fugiat maxime illum optio iure ..."

date: 1998-08-22T11:45:03.000+00:00

_id: ObjectId('5a9427648b0beeb6b69579d3')

name: "Cameron Duran"

email: "cameron_duran@fakegmail.com"

movie_id: ObjectId('573a1390f29313caabdc4171')

Movies # 24k

cluster27300.b79ykvq...

users

comments

movies

sessions

theaters

mongodb: cluster27300...

cluster27300.b79ykvq.mongodb.net

arths

movies

Documents24K

Aggregations

Schema

Indexes1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ExplainResetFindOptions

ADD DATAUPDATEDELETEEXPORT DATAEXPORT CODE

251 - 25 of 23539

_id: ObjectId('573a1390f29313caabdc4135')

plot: "Three men hammer on an anvil and pass a bottle of beer around."

genres: Array (1)

runtime: 1

cast: Array (2)

num_oflix_comments: 1

title: "Blacksmith Scene"

fullplot: "A stationary camera looks at a large anvil with a blacksmith behind it..."

countries: Array (1)

released: 1893-05-09T00:00:00.000+00:00

directors: Array (1)

rated: "UNRATED"

awards: Object

lastupdated: "2015-08-26 00:03:50.133000000"

year: 1893

imdb: Object

type: "movie"

tomatoes: Object

_id: ObjectId('573a1390f29313caabdc42e8')

plot: "A group of bandits stage a brazen train hold-up, only to find a determ..."

genres: Array (2)

runtime: 11

Array (4)

https://m.media-amazon.com/images/M/HV5BMTU3MjESNzYTYTYyNS00MDVmLWlWYj..."

"The Great Train Robbery"

fullplot: "Among the earliest existing films in American cinema - notable as the ..."

Sessions #1

Theaters #1.6k

1564k

cluster27300.b79ykvq.mongodb.net > arths > theaters

Documents 1.6K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

25 1 - 25 of 1564

```
_id: ObjectId('59a47286cfa9a3a73e51e72c')
theaterId: 1000
location: Object
```

```
_id: ObjectId('59a47286cfa9a3a73e51e72d')
theaterId: 1003
location: Object
```

```
_id: ObjectId('59a47286cfa9a3a73e51e72e')
theaterId: 1008
location: Object
```

```
_id: ObjectId('59a47286cfa9a3a73e51e72f')
theaterId: 1004
location: Object
```

```
_id: ObjectId('59a47286cfa9a3a73e51e730')
theaterId: 1002
location: Object
```

```
_id: ObjectId('59a47286cfa9a3a73e51e731')
```

```
/*sessions.json*/
```

```
{
  "id": {"$oid": "5a97f9c91c807bb9c6eb5fb4"},
  "user_id": "t3qulfeem@kwiv5.6ur",
  "jwt":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpYXQiOiJlMTk5MDkzMjEsIm5iZiI6MTUxOTkwOTMyMSwianRpIjoiaWJlZDAwMWYtNTFiYi00NzVhLTgxZDA0MDcwNGE5Mjk0MWZlIiwiaXNjaXNTE5OTEwMjI0LCJpZGVudGl0eSI6eyJlbWVpYyJlbnRd2l2NS42dXIiLCJuYXZlIjoim2lveHJtZnF4IiwicGFzc3dvcmQiOiOm51bGx9LCJmcmVzaCI6ZmFsc2UsInR5cGUiOiJhY2Nlc3MiLCJ1c2VyX2NsYWltcyI6eyJlYyJlbnRd2l2NyZCI6bnVsbH19fQ.ejtr_NyZyBronWMKuE0RFTjWej-T0zGrdc_ymGtVs"
}
```

/* movies.json */

_id, Title, year, runtime, Genres, Cast, directors, plot, fullplot, Countries, Languages, rated, released_timestamp, imdb_rating, imdb_votes, imdb_id, num_mflix_comments, poster, Lastupdated, Tomatoes_viewer_rating, Tomatoes_viewer_reviews, Tomatoes_viewer_meter

573a1390f29313caabcd4135, blacksmith Scene, 1893, 1, Short,"Charles Kayser, John Ott",William K.L. Dickson, Three men hammer on an anvil and pass a bottle of beer around., "A stationary camera looks at a large anvil with a blacksmith behind it and one on either side. The smith in the middle draws a heated metal rod from the fire, places it on the anvil, and all three begin a rhythmic hammering. After several blows, the metal goes back in the fire. One smith pulls out a bottle of beer, and they each take a swig. Then, out comes the glowing metal and the hammering resumes.", uSA,, uNRATED, -2418768000000,6.2, 1189, 5, 1,, 2015-08-26 00:03:50.133000000, 3, 184, 32

573a1390f29313caabcd42e8, The Great Train Robbery, 1903, 11,"Short, Western", "A.C. Abadie, Gilbert M. 'Broncho Billy' Anderson, George Barnes, Justus D. Barnes", edwin S. Porter,"A group of bandits stage a brazen train hold-up, only to find a determined posse hot on their heels.", "Among the earliest existing films in American cinema - notable as the first film that presented a narrative story to tell - it depicts a group of cowboy outlaws who hold up a train and rob the passengers. They are then pursued by a Sheriff's posse. Several scenes have color included - all hand tinted.", uSA, english, TV-G, -2085523200000, 7.4, 9847, 439,0, <https://media-amazon.com> 00:27:59.177000000, 3.7, 2559, 75

/* comments.json */

_id, name, email,movie_id, Text, date_timestamp

5a9427648b0beebeb69579d0, Talisa Maegyr, oona_chaplin@gameofthron.es , 573a1390f29313caabcd41b1,"Rem itaque ad sit rem voluptatibus. Ad fugiat maxime illum optio iure alias minus. Optio ratione suscipit corporis qui dicta.", 903786303000

5a9427648b0beebeb69579d3, Cameron Duran, Cameron_duran@fakegmail.com , 573a1390f29313caabcd4217,Quasi dicta culpa asperiores quaerat perferendis neque. Est animi pariatur impedit itaque exercitationem., 420323955000

/* users.json */

_id, name, email, password

59b99db4cfa9a34dcd7885b6, ned Stark, Sean_bean@gameofthron.es, \$2b\$12\$UREFwsRUoyF0CRqGNK0LzO0HM/jLhgUCNNIJ9RJAqMUQ74crlJ1Vu

59b99db4cfa9a34dcd7885b7, Robert Baratheon, mark_addy@gameofthron.es , \$2b\$12\$yGqxLG9LZpXA2xVDhuPnSOZd.VURVkv7wgOLY3pnO0s7u2S1ZO32y

→ la FECHA (59b99db4): código hexadecimal. Si los convertimos a texto normal: **13 de septiembre de 2017 a las 21:05:56 UTC** → 2017-09-13

first: 2017-09-13 21:05:56

/*theaters.json */

_id, TheaterId, Street1, City, State, zipcode, Geo_type, Longitude, Latitude

59a47286cfa9a3a73e51e72c, 1000, 340 W Market, bloomington,MN, 55425, point, -93.24565, 44.85466

59a47286cfa9a3a73e51e72d, 1003, 45235 Worth Ave., California,MD, 20619, point, -76.512016, 38.29697

NIVEL 1 – Fundamentos de Ingesta y Consultas Básicas

N1.1 Ingesta de datos (Data Ingestion)

Crear la base de datos e importar las colecciones desde archivos JSON adjuntos.

MongoDB Compass para cargar los documentos en crudo (**Bronze Layer**).

Observar, cuestionar la integridad de esquemas flexibles y tipos de datos (como arrays y campos geoespaciales).

N1.1.1. Mostrar los 2 primeros comentarios

```
// N1.1.1: los 2 primeros comentarios .....
db.comments.find().limit(2) //find() (busca todos) + .limit(2) (frena en el 2o)
```

```
> // N1.1.1: los 2 primeros comentarios .....
db.comments.find().limit(2) //find() (busca todos) + .limit(2) (frena en el 2o)
< {
  _id: ObjectId('5a9427648b0beebe69579cc'),
  name: 'Andrea Le',
  email: 'andrea_le@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd418c'),
  text: 'Rem officiis eaque repellendus amet eos doloribus. Porro dolor voluptatum voluptates neque culpa molestias. Voluptate unde nulla +',
  date: 2012-03-26T23:20:16.000Z
}
{
  _id: ObjectId('5a9427648b0beebe69579cf'),
  name: 'Greg Powell',
  email: 'greg_powell@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd41b1'),
  text: 'Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Commodi nisi sit placeat rerum vero cupiditate neque. Dolorum',
  date: 1987-02-10T00:29:36.000Z
}
Atlas atlas-2i6tp4-shard-0 [primary] arths>
```

```
  _id: '20002',
  total: 1
}
> // 1. Los 2 primeros comentarios
db.comments.find().limit(2).pretty()
< {
  _id: ObjectId('5a9427648b0beebe69579cc'),
  name: 'Andrea Le',
  email: 'andrea_le@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd418c'),
  text: 'Rem officiis eaque repellendus amet eos doloribus. Porro dolor voluptatum voluptates neque culpa molestias. Voluptate unde nulla temporibus ullam.',
  date: 2012-03-26T23:20:16.000Z
}
{
  _id: ObjectId('5a9427648b0beebe69579cf'),
  name: 'Greg Powell',
  email: 'greg_powell@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd41b1'),
  text: 'Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Commodi nisi sit placeat rerum vero cupiditate neque. Dolorum nihil vero animi.',
  date: 1987-02-10T00:29:36.000Z
}
```

N1.1.2. Contar usuarios registrados

// N1.1.2: ¿cuántos usuarios registrados hay?

db.users.countDocuments({}) // cuenta todos los docs (Llaves VACIAS = sin , filtro vacio) // 185

```
> // N1.1.2: ¿cuántos usuarios registrados hay? .....  
db.users.countDocuments({}) // countDocuments({}) cuenta todos los docs ( Llaves VACIAS = sin , filtro vacio)  
< 185  
Atlas atlas-2i6tp4-shard-0 [primary] arths>
```

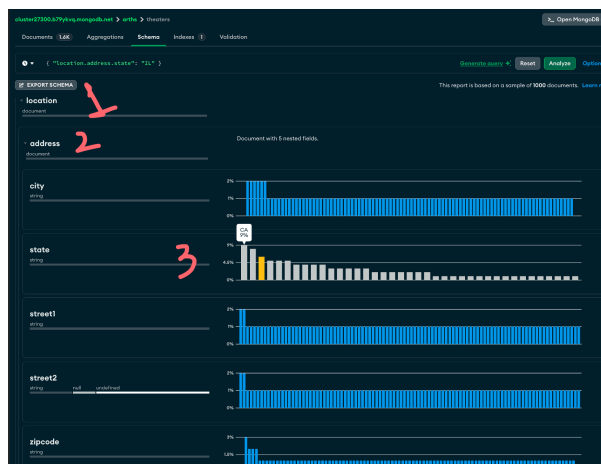
Calculo y obtención del número de Users. En suma, métrica básica de volumen (tamaño total base) – Cardinalidad de entidad usuario → Número de elementos únicos).

N1.1.3. Contar cines en el ESTADO (State) California.

Para familiarizarse con los datos :→ **Schema** → **"Analyze Schema"**.

Compass escanea los casi 1.6k cines y genera un informe visual completo. → los campos ("columnas") y un gráfico de barras con los valores más comunes.

Al desplegar location -> address -> state, el gráfico muestra la barra de "CA", permitiendo así identificar como se denomina al Estado solicitado.

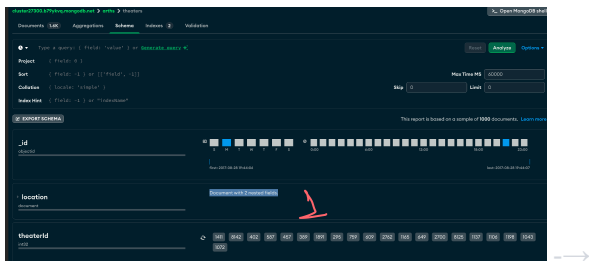


*Al parecer, en contexto MDB, los datos también se ven condicionados por lo denominado como **campos anidados** (o nested/embedded documents), es decir, la capacidad de almacenar documentos dentro de otros documentos.*

*A diferencia de las bases de datos relacionales, donde se separa la información en varias tablas y se tienen que unir entre JOINS relacionales, MongoDB permite agrupar datos relacionados en una **estructura jerárquica**.*

*Entonces, esto mismo no solo se refleja en la forma de llegar a los datos, la ruta vía 3 niveles, sino también mediante la **Notación de Puntos (Dot Notation)**, pues, para acceder a un valor "profundamente" anidado, se utiliza la notación de puntos. Lo cual, por cierto, de este mismo principio de anidación u caso Array, deberá accederse mediante **Comillas** (" ").*

Ergo, DENTRO de las Llaves de una consulta {}, si hay un punto (.), tienen que haber comillas obligatoriamente.



Document with 2 nested fields. -> Niveles "contados" en el gráfico anterior.

```
// N1.1.3: ¿cuántos cines hay en California? .....
db.theaters.countDocuments({ "location.address.state": "CA" }) // 169
```

```
> // N1.1.3: ¿cuántos cines hay en California? .....
  db.theaters.countDocuments({ "location.address.state": "CA" })
< 169
Atlas atlas-2i6tp4-shard-0 [primary] arths>
```

Filtrado por campo anidado/estado

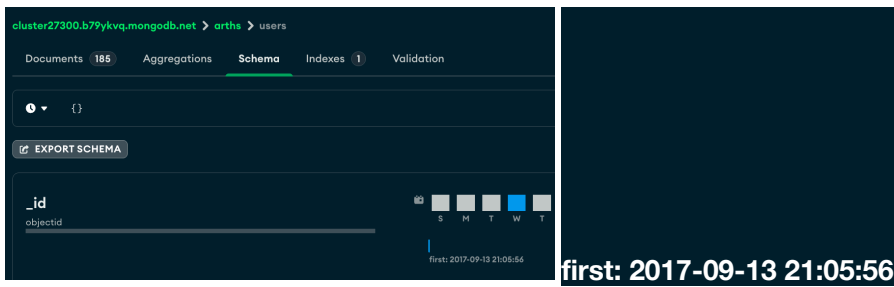
N1.1.4. Identificar el primer usuario registrado

// N1.1.4: ¿quién fue el primer usuario en registrarse?

db.users.find().sort({ _id: 1 }).limit(1) // _id lleva el timestamp de creación // sort({_id:1 =ASCENDIENTE }) ordena del más antiguo al más nuevo

Los ObjectId se generan automáticamente de forma cronológica. Por tanto, ordenando por _id de forma ascendente conseguimos el primer registro. Ergo, _id (aka ObjectId) lleva timestamp el cual nos permite tomar el 1r dato grabado.

.sort({ _id: -1 }): orden DESCENDENTE.

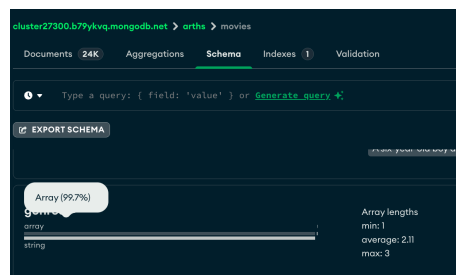


```
> // N1.1.4: ¿quién fue el primer usuario en registrarse? .....
  db.users.find().sort({ _id: 1 }).limit(1)
< {
  _id: ObjectId('59b99db4cfa9a34dcd7885b6'),
  name: 'Ned Stark',
  email: 'sean_bean@gameofthron.es',
  password: '$2b$12$UREFwsRUoyF0CRqGNK0Lz00HM/jLhgUCNNIJ9RJAqMUQ74cr1J1Vu'
}
```

Uso de sort() + limit(1) – Patrón de obtención de registro más antiguo, ordenado y limit 1.

N1.1.5. Contar películas de género comedia.

Nuevamente, familiarizándose con los datos en MongoDB:



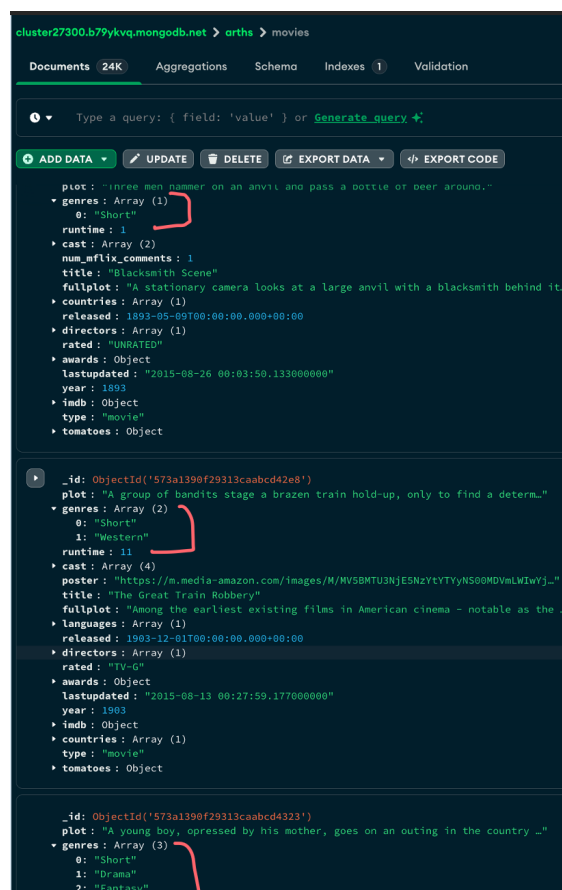
Array vs String

Array lengths 99.7% = Porcentaje de los Datos con ese formato de lista (Estadísticas de la Lista).

- min: 1
- average: 2.11
- max: 3

Además, en "Documents" → Al desplegar Genres, se puede comprobar también que es un **Array**.

genres: Array (2) → La lista numerada : 0: "Short", 1: "Western"



En este caso, prima la **Mecánica Multi-key Indexing / Querying**; pues, como genres es un array, MongoDB busca dentro de la lista de forma nativa sin tener que usar \$unwind en RAM ni normalizar con tablas puente N:M de SQL u operaciones costosas de desanidado (\$unwind o UNNEST en SQL); el motor desglosa el **índice B-tree** apuntando al mismo `_id` para resolver la búsqueda en tiempo constante.

// N1.1.5: ¿cuántas películas de comedia hay?

db.movies.countDocuments({ genres: "Comedy" }) // genres es un array; buscar un texto casa si el array lo tiene // 7024

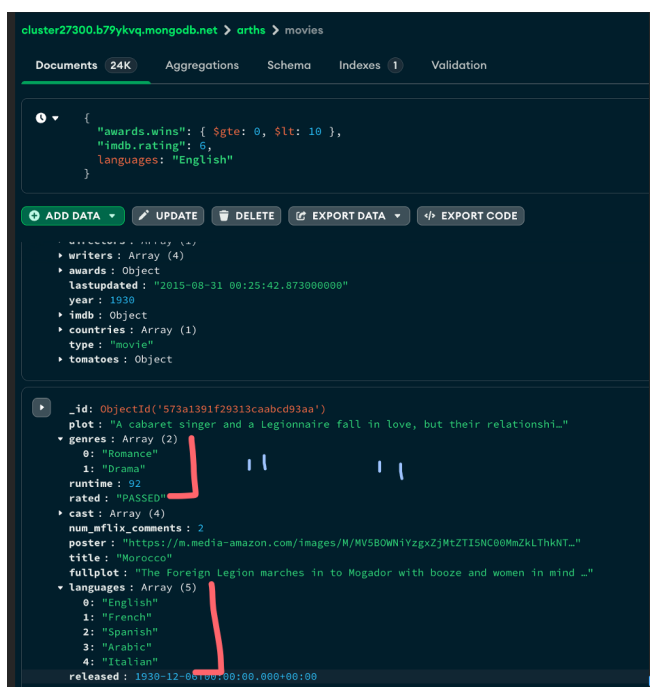
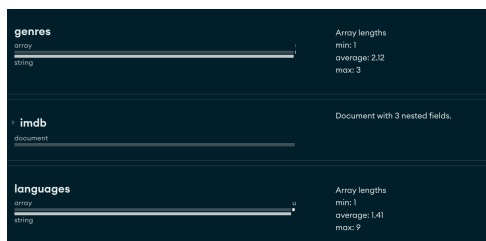
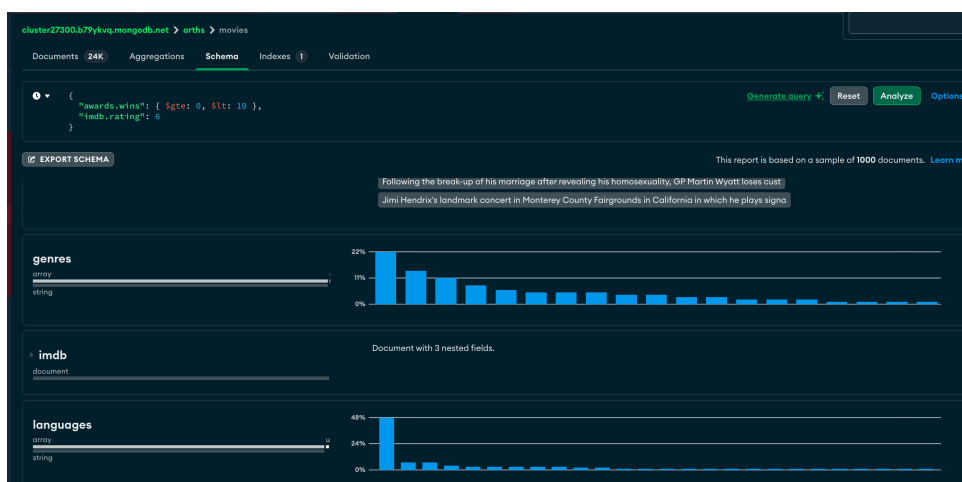
```
> // N1.1.5: ¿cuántas películas de comedia hay? .....
db.movies.countDocuments({ genres: "Comedy" }) // genres es un array; buscar un texto casa si el array lo tiene
< 7024
```

N1.2 – Filtros compuestos Mostrar películas producidas en 1932 donde el género sea drama o el idioma sea francés.

Uso de operadores lógico, \$or, filtros en arrays y campos de texto. Tanto el Género como el Idioma son **arrays** (como "tipo de base", pero, los elementos son **String**. Es decir, que, para el motor de almacenamiento de MongoDB, ambos campos son lo mismo; vectores de **cadena de caracteres**.

Así pues, siendo Strings, la lectura mecánica será de **Coincidencia Exacta** (Exact Value Match) por lo que, la búsqueda no será vía Búsqueda de Texto Inteligente (Text Search), siendo así, sensible (Case Sensitivity) y, en caso de diferencias de escritura (p.ej, "french" en Minúscula, en otro idioma ("Francés"), con algún espacio u demás distinción textual) se excluirían estos datos de la operación y, por tanto, podría suscitar un riesgo de Integridad de Datos.

Si bien la situación no lo amerita -entiendo, especulo- convendría en otra disyuntiva el uso de alguna operación Index.



N1.3 – Rangos y múltiples condiciones. Películas estadounidenses con entre 5 y 9 premios, producidas entre 2012 y 2014.

```
// N1.3: películas USA 5-9 premios, hechas entre años 2012 y 2014 (incluidos) .....
db.movies.find({
  countries: "USA",
  "awards.wins": { $gte: 5, $lte: 9 },
  year: { $gte: 2012, $lte: 2014 }})
```

(Uso de \$gte, \$lte, \$and implícito y filtros en campo awards numérico).

```
>_MONGOSH
> // N1.3: películas USA 5-9 premios, producidas entre años 2012 y 2014 (incluidos) .....
db.movies.find({
  countries: "USA",
  "awards.wins": { $gte: 5, $lte: 9 },
  year: { $gte: 2012, $lte: 2014 }
})
< {
  _id: ObjectId('573a13acf29313caabd29366'),
  fullplot: "The manager of the negative assets sector of Life magazine, Walter Mitty, has been working for sixteen years for the magazine and has a tedious life, not going any",
  imdb: {
    rating: 7.4,
    votes: 211230,
    id: 359950
  },
  year: 2013,
  plot: 'When his job along with that of his co-worker are threatened, Walter takes action in the real world embarking on a global journey that turns into an adventure more ext',
  genres: [
    'Adventure',
    'Comedy',
    'Drama'
  ],
  rated: 'PG',
  metacritic: 54,
  title: 'The Secret Life of Walter Mitty',
  lastupdated: '2015-08-31 00:10:51.747000000',
  languages: [
    'English',
    'Spanish',
    'Icelandic'
  ],
  writers: [
    'Steve Conrad (screenplay)',
    'Steve Conrad (screen story by)',
    'James Thurber (based on the short story by)'
  ],
  type: 'movie',
  tomatoes: {
    website: 'http://WalterMitty.com',
    viewer: {
      rating: 3.7,
      numReviews: 77663,
      meter: 70
    }
  },
}
```

Operadores de Consulta (Manipulación y Acotado)

- **Comparación:**
 - \$gt / \$gte: Mayor que / Mayor o igual que.
 - \$lt / \$lte: Menor que / Menor o igual que.
 - \$ne: No igual a.

NIVEL 2 – Agregaciones y Análisis por Dimensión

N2.1 Contar cuántos comentarios ha escrito un usuario cuyo email pertenece a dominio (Juego de Tronos)

- *Versión directa (countDocuments):* [query] → resultado X
Versión pipeline (más escalable): [query aggregate] → mismo resultado

```
// N2.1: comentarios escritos por usuarios con dominio @gameofthron.es .....
db.comments.countDocuments({
  email: { $regex: "@gameofthron\\.es$", $options: "i" }}) // $regex busca patrón; \. escapa el
punto [2 barras inclinadas] (el Punto Regex =cualquier caracter); | es$ (dolar)= final del texto
// $options:"i" = case-insensitivity →ignora mayúsculas/minúsculas // 22841
```

```
> // N2.1: comentarios escritos por usuarios con dominio @gameofthron.es .....
db.comments.countDocuments({
  email: { $regex: "@gameofthron\\.es$", $options: "i" }
}) // $regex busca patrón; \. escapa el punto [2 barras inclinadas] (el Punto Regex =cualquier caracter); | es$ (dolar)= final del texto
// $options:"i" = case-insensitivity →ignora mayúsculas/minúsculas
< 22841
```

Tras consulta, observo que, mi manera de plantear el ejercicio, ha sido una vía más directa (countDocuments, método de Conteo). Si bien esta, ha sido útil, se trataría de una forma sencilla de **envolver (wrapper)** el código complejo "susceptible" a ser realizado aquí; una **capa de personalización o un "intermediario" (Capa de Abstracción)** "escondida" en el **Filtro de Consulta (Query Filter)** -el objeto JSON con criterios de selección (las condiciones entre llaves { }) que se introduce dentro del conteo.

Por tanto, prosiguiendo, si bien es útil que optimice la legibilidad y complejidad al simplificar las operaciones CRUD (Create, Read, Update, Delete -- Crear, Leer, Actualizar, Borrar), dada sus limitaciones (como, la no posibilidad de, realizar más operaciones u cálculos), contrariamente, se puede operar mediante un **Pipeline de Agregación** (aggregate).

Este framework, **Aggregation Pipeline (canalización de agregación)**, a diferencia del anterior, es un Framework **secuencial** caracterizado por ser un proceso **modular** cuyos Bloques/Capas/Etapas (Stages) son las fases en las cuales, en cada una se va dando una Operación la cual, traslada el resultado a la siguiente fase y, por tanto, no pone en riesgo la integridad de las operaciones/resultados previos. Es decir, que ese flujo secuencial, permite aislar la complejidad facilitando así el mantenimiento, depuración y la reutilización de cada bloque de código de manera independiente (mayor modularidad).

```
// .....
db.comments.aggregate([
  {$match: {email: { $regex: "@gameofthron\\.es$", $options: "i" }}}, // Etapa 1: Filtrar documentos x patrón email

  {$count: "comentarios_totales" // Alias
  }
]) // E2: Contar resultados y nombre al campo de salidas //22841
```

```
> // .....
db.comments.aggregate([
  {$match: {email: { $regex: "@gameofthron\\.es$", $options: "i" }
  }
}, // Etapa 1: Filtrar documentos x patrón email

  {$count: "comentarios_totales"
  }
]) // E2: Contar resultados y nombre al campo de salidas
< {
  comentarios_totales: 22841
}
```

(Uso de agregación \$match con regex en email + \$count o \$group –**Data Quality** y segmentación de usuarios).

N2.2 Distribución de cines por código postal en Washington D.C

(Agregación \$group por campo location.zipcode + \$sum – Preparación de datos para dashboard / reporting).

```
// N2.2: ¿cuántos cines hay por código postal dentro del estado DC? .....  
//aggregate = tubería de pasos ($match → $group → $sort) ... Pipeline  
db.theaters.aggregate([  
  { $match: { "location.address.state": "DC" } }, // 1o filtra ....kinda WHERE SQL  
  { $group: { _id: "$location.address.zipcode",  
             total: { $sum: 1 } } // 2o agrupa, y cuenta +1 por cada cine (en el total)... Alias  
  }},  
  { $sort: { total: -1 } } // (-1 = DESC) 3a  
])
```

La Habiendo establecido las "bases" u precedente en el anterior ejercicio, la Canalización de Agregación de ahora, se ve marcada por 3 fases.

En **la 1ª Etapa de Segmentación y Limpieza de Datos (\$match)**, se **filtra**, acotando así a solo los documentos cuyo Estado sea Washington DC y, por ende, optimizando el rendimiento de sistema (CPU, Performance) al no dar pie al procesamiento innecesario de cientos de datos.

En **la 2ª Fase de Agregación y de Análisis de Dimensión ¿? (\$group)**.

el campo `_id`, obligatorio, aquí define la **dimensión o categoría de análisis**, que, da al asignarle la ruta `"$location.address.zipcode"`, indica a MongoDB que cree una partición por cada código postal único. Todos los cines que compartan el mismo código postal se consolidarán dentro del mismo bloque, y, que, consecuentemente, la métrica calcula (\$sum) cuente y acumule cada doc que entre bajo el alias "total", funcionando, así como un contador que reduce la granularidad del dataset original (basado en cines individuales) para transformarlo en valores agregados (totales por zona).

Y, finalmente, la Etapa 3, Ordenación Analítica (\$sort), en la que se marca la Direccionalidad/Sentido de la presentación de datos (de mayor a menor número de cines).

```
< 22841  
> // N2.2: ¿cuántos cines hay por código postal dentro del estado DC? .....  
//aggregate = tubería de pasos ($match → $group → $sort)  
db.theaters.aggregate([  
  { $match: { "location.address.state": "DC" } }, // 1º filtra ...kinda WHERE  
  { $group: { _id: "$location.address.zipcode",  
             total: { $sum: 1 } } // agrupa, y cuenta +1 por cada cine (en el total)  
  }},  
  { $sort: { total: -1 } } // (-1 = descendente)  
])  
< {  
  _id: '20010',  
  total: 1  
}  
{  
  _id: '20016',  
  total: 1  
}  
{  
  _id: '20002',  
  total: 1  
}
```

- Estado: location → address → state: **"location.address.state"**.

-Código Postal: location → address → zipcode: **"location.address.zipcode"**.

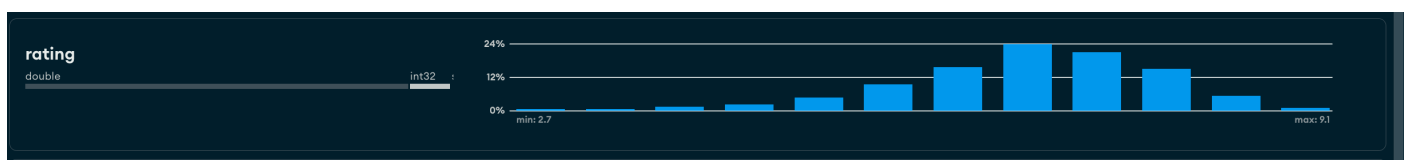
NIVEL 3 – Consultas Avanzadas y Visualización Geoespacial

N3.1 Películas dirigidas por John Landis con rating IMDb entre 7.5 y 8.0.

```
// N3.1: películas de John Landis + puntuación IMDb entre 7.5 y 8 .....
db.movies.find({
  directors: "John Landis",          // directors es array
  "imdb.rating": { $gte: 7.5, $lte: 8 }
})
```

```
> // N3.1: películas de John Landis con puntuación IMDb entre 7.5 y 8 .....
db.movies.find({
  directors: "John Landis",          // directors es array
  "imdb.rating": { $gte: 7.5, $lte: 8 }
})
< {
  _id: ObjectId('573a1397f29313caabce6d94'),
  fullplot: "Faber College has one frat house so disreputable it will take anyone. It has a second one full of white, anglo-saxon, rich young men who are so sanctimonious no one can stand
  imdb: {
    rating: 7.6,
    votes: 84834,
    id: 77975
  },
  year: 1978,
  plot: 'At a 1962 college, Dean Vernon Wormer is determined to expel the entire Delta Tau Chi Fraternity, but those trouble-makers have other plans for him.',
  genres: [
    'Comedy'
  ],
  rated: 'R',
  metacritic: 82,
  title: 'Animal House',
  lastupdated: '2015-09-13 00:02:47.803000000',
  languages: [
    'English',
    'Italian'
  ],
  writers: [
    'Harold Ramis',
    'Douglas Kenney',
    'Chris Miller'
  ],
  type: 'movie',
  tomatoes: {
    website: 'http://www.animalhouse.com/',
    viewer: {
      rating: 3.8,
      numReviews: 184490,
      meter: 89
    },
    dvd: 1998-02-24T00:00:00.000Z,
    critic: {
      rating: 7.7,
      numReviews: 46,
```

(Filtro en array *directors* + rango en campo embebido *imdb.rating* – Optimización con índices compuestos recomendada).



A nivel arquitectura, el campo `imdb.rating` presenta **inconsistencia de tipos** entre documentos (Double vs Int32), por lo que, para trabajar con esto, el motor de base de datos ejecuta **coerción de tipos** (convierte automáticamente un tipo de dato en otro de forma temporal para poder hacer una operación) ... Entonces, si bien funciona, esto no significa que los datos sean lo más "sanos" posibles. Por ello, para garantizar la fiabilidad del dato y no arriesgar la integridad del esquema, conviene aislar un riesgo "latente" como este y, pero en un pipeline de producción convendría normalizar con `$convert` por ejemplo.

N3.2 Visualización Geoespacial Mostrar en mapa la ubicación de todos los teatros (theatres).

```
// N3.2: mostrar en un mapa la ubicación de todos los teatros .....
db.theaters.createIndex({ "location.geo": "2dsphere" })
// crea INDICE GEOESPACIAL 2dsphere sobre las coordenadas

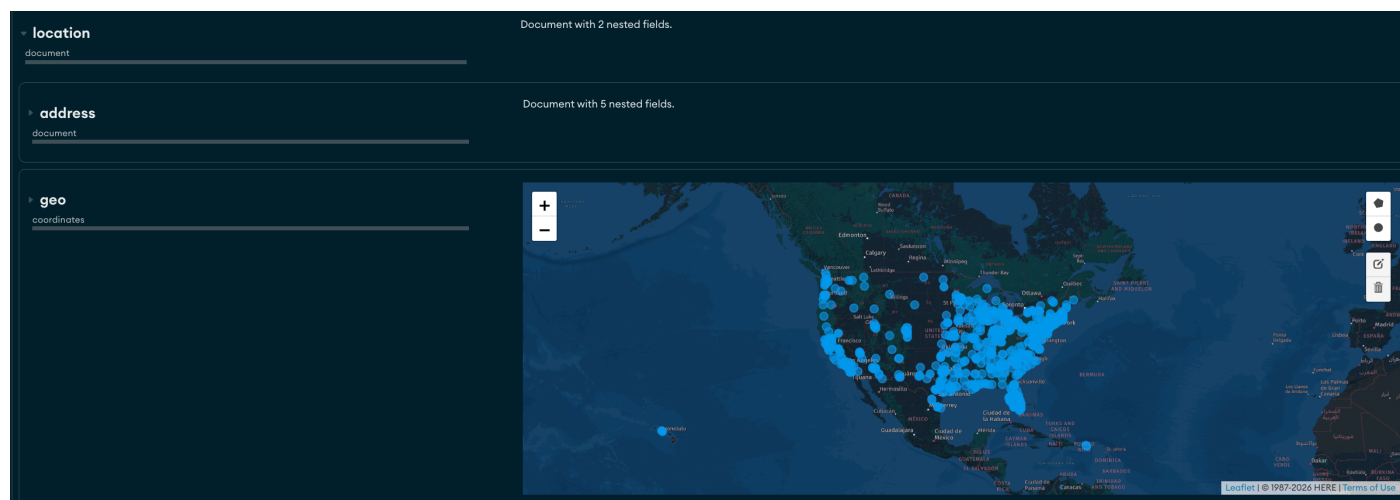
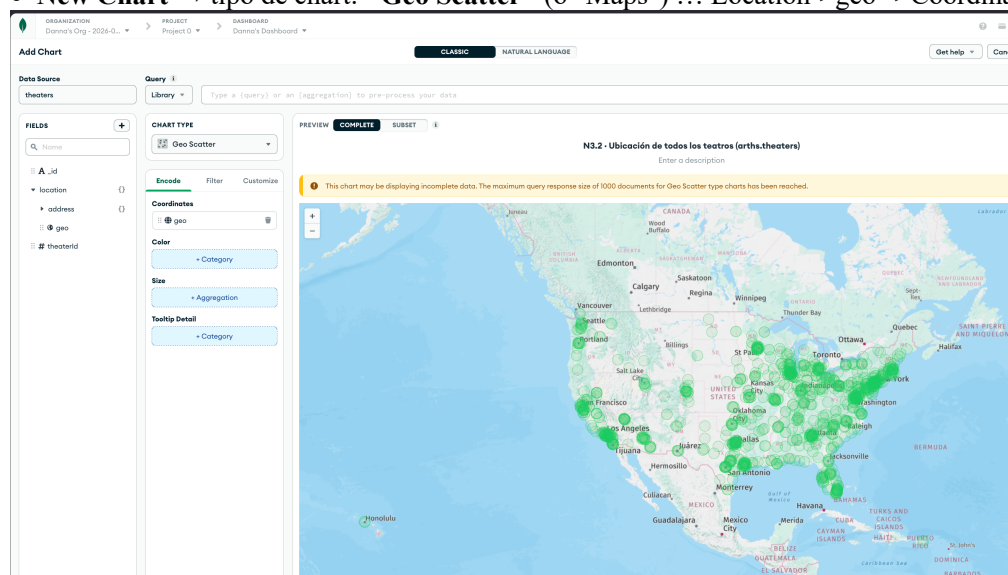
// o
db.theaters.createIndex({ "location.geo.coordinates": "2dsphere" })
```

```
> // N3.2: mostrar en un mapa la ubicación de todos los teatros .....
db.theaters.createIndex({ "location.geo": "2dsphere" })
// crea INDICE GEOESPACIAL 2dsphere sobre las coordenadas

//
db.theaters.createIndex({ "location.geo.coordinates": "2dsphere" })
< location.geo.coordinates_2dsphere
Atlas atlas-2i6tp4-shard-0 [primary] arths>
```

Desde la página web de Atlas → Services → Visualization

- cloud.mongodb.com → "tu proyecto" → "Charts" -> **Charts Builder**
- **Add Data Source** → clúster cluster27300 → colección arths.theaters
- **New Chart** → tipo de chart: "Geo Scatter" (o "Maps") ... Location->geo-->Coordinates



(Uso de índice 2dsphere + \$geoNear o exportación de coordenadas para visualización en Compass / MongoDB Charts / Power BI / Tableau – **Spatial Analytics**).

EXTRA: Apuntes problemática experimentada (my own manual):

Incidencias y Conectividad MongoDB Atlas → Pérdida de Conexión

MongoServerError + TLSV1_ALERT_INTERNAL_ERROR.

- **Causa raíz:** Al movilizarme, cambiar de Conexión, Wifi, IP. El clúster de MongoDB Atlas rechazó la conexión porque la IP pública de la cafetería no estaba autorizada en la lista blanca de seguridad (*Firewall*).

```
Type "it" for more
// N2.1: comentarios escritos por usuarios con dominio @gameofthron.es .....
db.comments.countDocuments({
  email: { $regex: "@gameofthron\\.es$", $options: "i" }
})
// $regex busca patrón; \\. escapa el punto [2 barras inclinadas] (el Punto Regex =cualquier caracter); | es$ (dolar)= final del texto
// $options:"i" = case-insensitivity ->ignora mayúsculas/minúsculas
MongoServerError: 1220039610752:error:10000438:SSL routines:OPENSSL_internal:TLSV1_ALERT_INTERNAL_ERROR:../third_party/boringssl/src/ssl/tls_record.cc:486:SSL alert number 86
// N2.1: comentarios escritos por usuarios con dominio @gameofthron.es .....
```

2. La Solución:

Ir Consola web de MongoDB Atlas (Network Access -> Add IP Access List Entry).

IP Access List

+ADD IP ADDRESS			
You will only be able to connect to your cluster from the following list of IP Addresses:			
IP Address	Comment	Status	Actions
0.0.0.0/0 (includes your current IP address)	Acces universal	Active	EDIT DELETE
89.141.215.243/32		Active	EDIT DELETE

<https://cloud.mongodb.com/v2/6a4b688c2fa4182921ad3c85#/security/network/accessList>

La regla de red **0.0.0.0/0** .

DA/DE: Esta notación CIDR representa un comodín global (máscara de red cero) => Significa "cualquier dirección IPv4 del mundo".

Elimina la necesidad de actualizar IP manualmente cada vez que cambie de localización, abriendo el puerto de escucha del clúster a cualquier petición externa autenticada.